

Dynamische Speicherallokation im Kernel

Ein Kernelmodul mit vollständigen Fehler- und Freigabepfaden entwickeln

3.08

Das Labor endet mit einem belastbaren Ressourcenvertrag

01 Allokieren

Pointer-Array, Objekte und Nutzdaten sicher reservieren.

02 Validieren

Parameter Grenzen und jede NULL-Rückgabe prüfen.

03 Zurückrollen

Teilallokationen in umgekehrter Reihenfolge freigeben.

04 Nachweisen

Erfolg, Grenzwerte und simulierte Fehler reproduzierbar testen.

Abnahmekriterium Jeder Ladeversuch endet entweder vollständig initialisiert oder vollständig aufgeräumt.

Das Demo-Modul besitzt drei Allokationsebenen

EBENE 1

Pointer-Array

```
struct demo_item **items
```

EBENE 2

Objekte

```
struct demo_item
```

EBENE 3

Payload je Objekt

```
u8 *payload
```

Warum drei Ebenen?

Ein Fehler nach Objekt 3 erzwingt einen echten partiellen Rückrollpfad.

Ownership koppelt Reservierung und Freigabe

Ressource	Erzeuger	Owner	Freigabe
items[]	demo_init()	Modul	kfree(items)
demo_item	item_create()	items[i]	kfree(item)
payload	item_create()	demo_item	kfree(item->payload)

Regel: Nur der festgelegte Owner gibt die Ressource frei – genau einmal.

Das Datenmodell macht Länge und Besitz explizit

```
struct demo_item {  
    u32 id;  
    size_t len;  
    u8 *payload;  
};  
  
static struct demo_item **items;  
static unsigned int item_count;
```

- 1** **id**
Eindeutige Kennung für Log und Test.
- 2** **len**
Gültige Größe des zugehörigen Payloads.

Der Pointer bleibt NULL, bis die jeweilige Allokation erfolgreich abgeschlossen ist.

Parameter vor der Allokation begrenzen

```
static uint count = 4;  
static uint payload_size = 128;  
  
module_param(count, uint, 0444);  
module_param(payload_size, uint, 0444);
```

```
if (!count || count > 1024)  
    return -EINVAL;  
  
if (!payload_size || payload_size >  
    PAGE_SIZE)  
    return -EINVAL;
```

Die Laborgrenzen sind bewusst konservativ und dürfen später begründet angepasst werden.

Keine Allokation mit ungeprüften, extern gesetzbaren Größenparametern starten.

kcalloc reserviert das Pointer-Array überlaufsicher

```
items = kcalloc(count, sizeof(*items), GFP_KERNEL);  
if (!items)  
    return -ENOMEM;
```

sizeof(*items) bleibt korrekt, wenn sich der Pointertyp später ändert.

01 Array-Helfer

count × Elementgröße wird auf Überlauf geprüft.

02 Nullinitialisierung

Noch nicht belegte Einträge beginnen als NULL.

03 Passender Kontext

Modulinitialisierung darf GFP_KERNEL verwenden.

item_create erzeugt nur vollständige Objekte

```
static struct demo_item *item_create(u32 id, size_t len)
{
    struct demo_item *item;

    item = kzalloc(sizeof(*item), GFP_KERNEL);
    if (!item)
        return NULL;

    item->payload = kmalloc(len, GFP_KERNEL);
    if (!item->payload)
        goto out_free_item;

    item->id = id;
    item->len = len;
    return item;

out_free_item:
    kfree(item);
    return NULL;
}
```

Die Funktion veröffentlicht nur vollständig erzeugte Objekte.

item_destroy gibt Payload und Objekt geordnet frei

```
static void item_destroy(struct demo_item *item)
{
    if (!item)
        return;

    kfree(item->payload);
    kfree(item);
}
```

01 payload
zuerst freigeben

02 demo_item
zuletzt freigeben

Idempotent für NULL – aber nicht für einen bereits freigegebenen, nicht-NULL Pointer.

Der Index beschreibt den Teilerfolg

```
for (i = 0; i < count; i++) {  
    items[i] = item_create(i, payload_size);  
    if (!items[i])  
        goto out_free_items;  
}  
  
item_count = count;  
return 0;
```

```
out_free_items:  
    while (i > 0) {  
        i--;  
        item_destroy(items[i]);  
    }  
    kfree(items);  
    items = NULL;  
    return -ENOMEM;
```

Damit bleibt der Rückrollpfad linear und überprüfbar.

Invariante

Vor jedem Schleifendurchlauf sind genau die Einträge 0 bis $i-1$ vollständig gültig.

module_init und module_exit teilen denselben Cleanup-Vertrag

```
static int __init demo_init(void)
{
    int ret = validate_params();

    if (ret)
        return ret;
    return create_items();
}
```

```
static void __exit demo_exit(void)
{
    while (item_count > 0) {
        item_count--;
        item_destroy(items[item_count]);
    }
    kfree(items);
}

module_init(demo_init);
module_exit(demo_exit);
```

Init-Fehler: create_items räumt auf. Erfolgreiches Init: demo_exit übernimmt die Ressourcen.

Bauen und Laden bilden den End-to-End-Test

```
obj-m += alloc_demo.o

all:
    $(MAKE) -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
    $(MAKE) -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

```
$ make
$ sudo insmod alloc_demo.ko count=8 \
payload_size=256

$ sudo dmesg | tail -n 20
$ sudo rmmod alloc_demo
$ sudo dmesg | tail -n 20
```

LOAD

8 vollständige Objekte

LOG

Parameter und Anzahl

UNLOAD

alle Ressourcen frei

Ein Laborparameter macht den Teilfehler deterministisch

```
static int fail_after = -1;  
module_param(fail_after, int, 0444);
```

```
for (i = 0; i < count; i++) {  
    if (fail_after >= 0 && i == fail_after)  
        goto out_free_items;  
  
    items[i] = item_create(i, payload_size);  
    if (!items[i])  
        goto out_free_items;  
}
```

Testfall

```
$ insmod alloc_demo.ko \  
count=8 \  
fail_after=3
```

Erwartung

Laden scheitert; Objekte 0–2 und das Array werden freigegeben.

Diese kontrollierte Simulation prüft den Modulcode ohne zufällige Nebeneffekte.

Die Testmatrix deckt Rückrollpfade ab

Aufruf	Erwartung	Prüfpunkt
count=4 payload_size=128	Laden erfolgreich	4 Objekte; Exit vollständig
count=0	-EINVAL	keine Allokation begonnen
payload_size=0	-EINVAL	keine Allokation begonnen
count=8 fail_after=3	-ENOMEM	Objekte 0–2 zurückgerollt
50× Laden/Entladen	immer erfolgreich	keine Warnung; stabiler Zustand

Optional: SLUB-/Fault-Injection-Debugging nur auf vorbereiteten Testsystemen aktivieren.

Vier Gewohnheiten verhindern die meisten Allokationsfehler

01 Vorher begrenzen

Externe Größen validieren und Überläufe vermeiden.

02 Owner benennen

Jede Ressource besitzt genau einen Freigabepfad.

03 Teilerfolg modellieren

Index und NULL-Zustände beschreiben gültige Ressourcen.

04 Fehler erzwingen

Rückrollpfade deterministisch und wiederholt ausführen.

Nächster Schritt

User-/Kernel-Schnittstellen: Daten kontrolliert zwischen beiden Welten übertragen.